

Logical Database Design and the Relational Model

Chapter # 05

Logical Database Design and the Relational Model

ER-to-Relational Mapping Algorithm

- Step 1: Mapping of Regular Entity Types***
- Step 2: Mapping of Weak Entity Types***
- Step 3: Mapping of Binary 1:1 Relation Types***
- Step 4: Mapping of Binary 1:N Relationship Types.***
- Step 5: Mapping of Binary M:N Relationship Types.***
- Step 6: Mapping of Multivalued attributes.***
- Step 7: Mapping of N-ary Relationship Types.***

Mapping EER Model Constructs to Relations

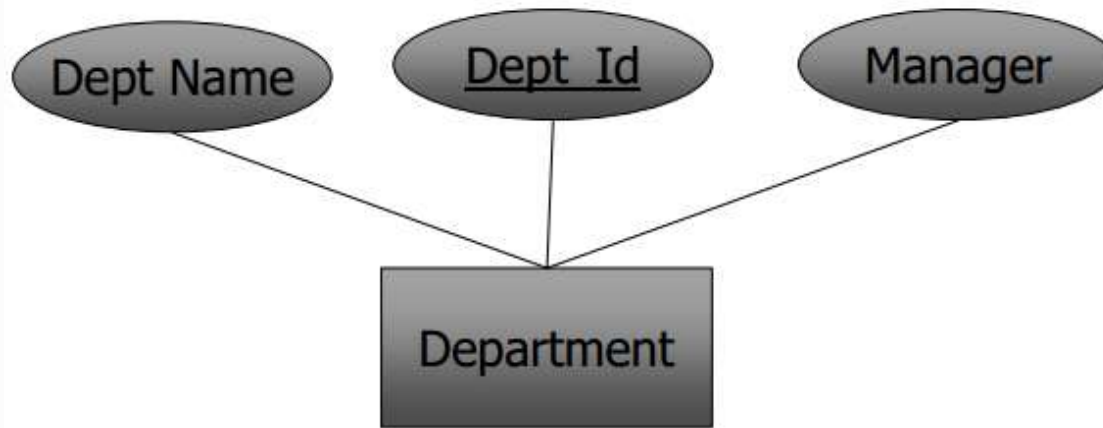
- Step 8: Options for Mapping Specialization or Generalization.***
- Step 9: Mapping of Union Types (Categories).***

Step 1: Mapping of Regular Entity Types.

For each regular (strong) entity type E in the ER schema, create a relation R that includes all the simple attributes of E .

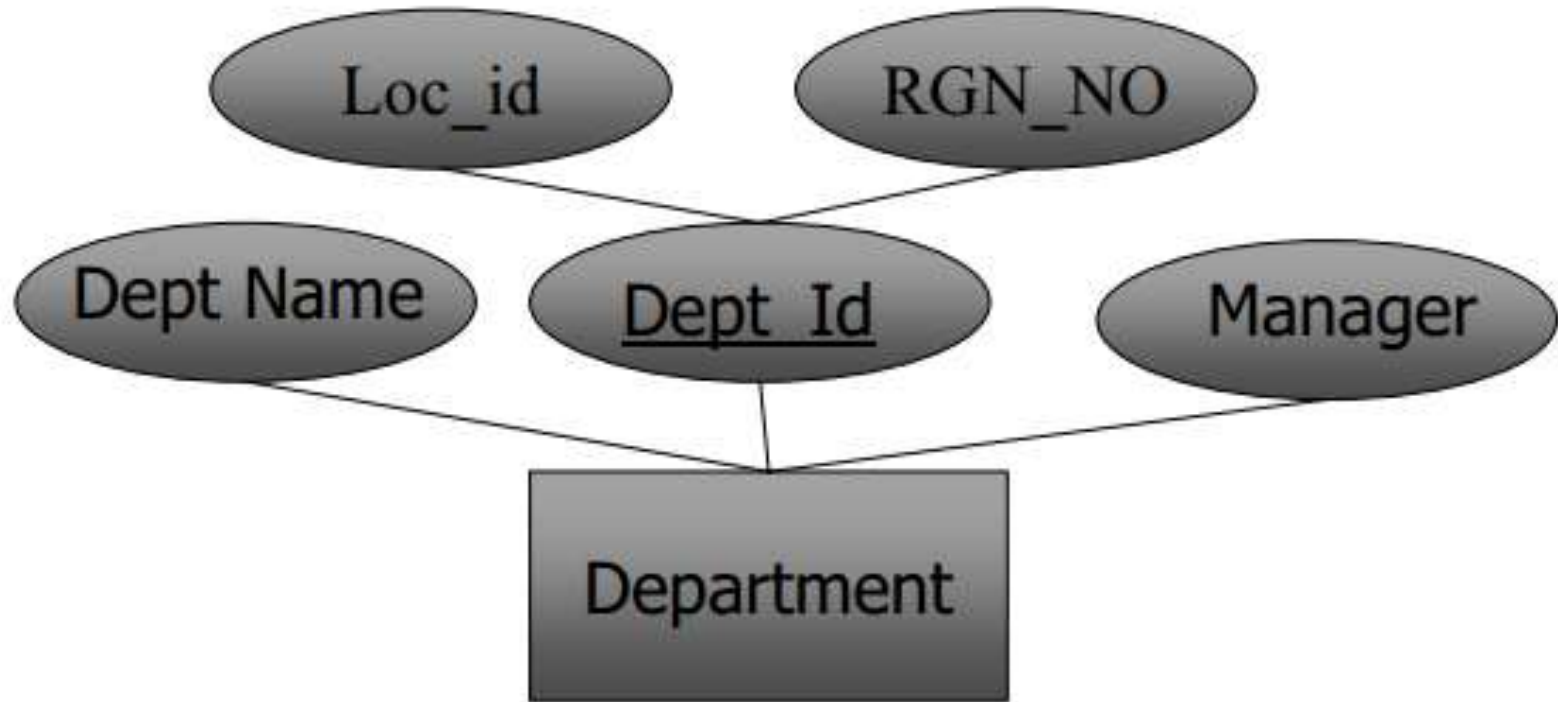
- Choose one of the key attributes of E as the primary key for R .
- If the chosen key of E is composite, the set of simple attributes that form it will together form the primary key of R

Step 1: Mapping of Regular Entity Types.



Department (Dept_id, DeptName, Manager)

Step 1: Mapping of Regular Entity Types.



Department (Loc_id, RGN_NO, DeptName, Manager)

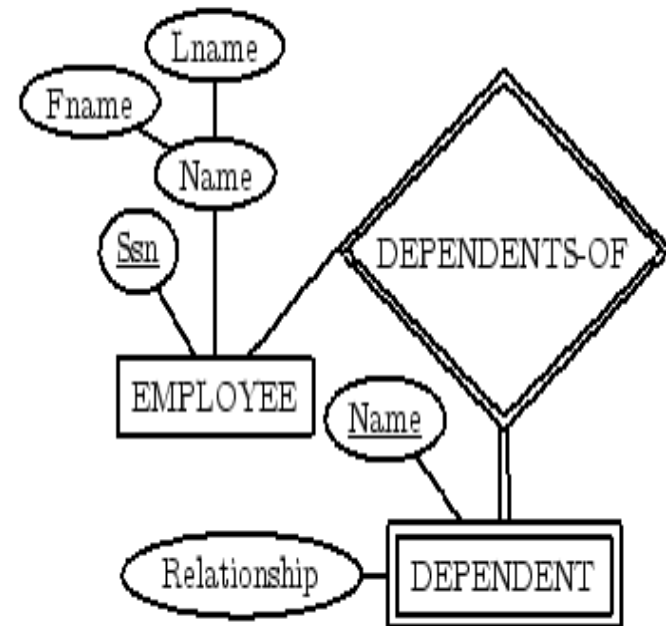
Step 2: Mapping of Weak Entity Types

For each weak entity type W in the ER schema with owner entity type E , create a relation R & include all simple attributes (or simple components of composite attributes) of W as attributes of R .

Also, include as foreign key attributes of R the primary key attribute(s) of the relation(s) that correspond to the owner entity type(s).

The primary key of R is the combination of the primary key(s) of the owner(s) and the partial key of the weak entity type W , if any.

Step 2: Mapping of Weak Entity Types



DEPENDENT	Relationship	<u>EMPL-SSN</u>	<u>NAME</u>
-----------	--------------	-----------------	-------------

Step 3: For each binary 1:1 relationship type R

Identify the relations S and T that correspond to the entity types participating in R. Choose one of the relations, say S, and include as foreign key in S the primary key of T.

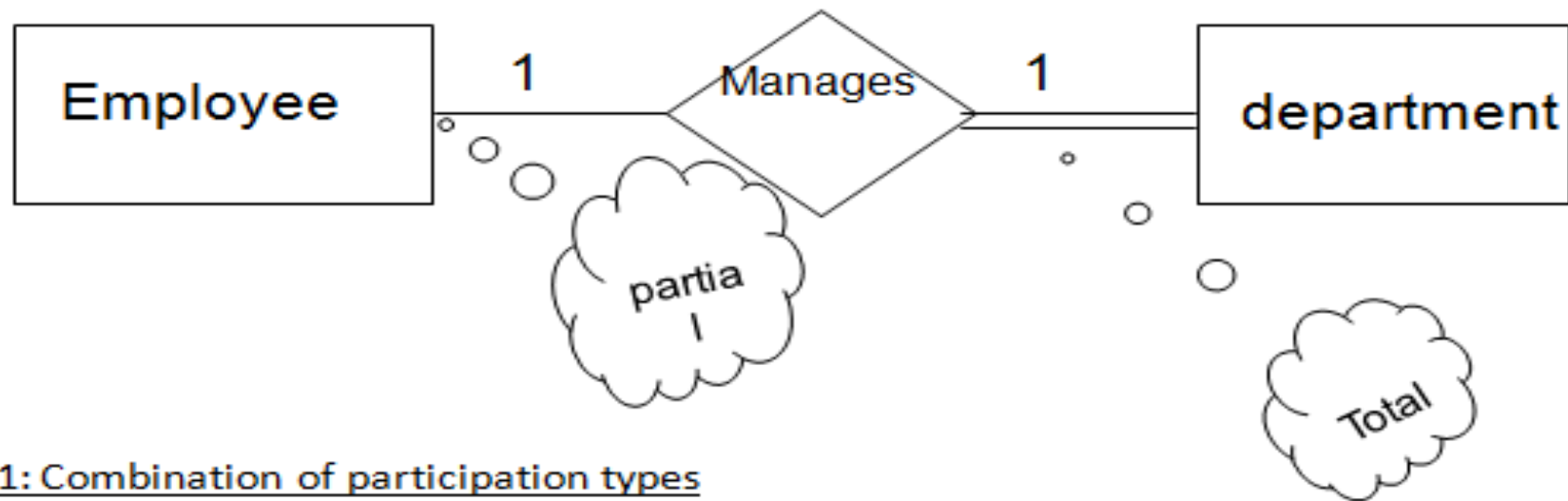
It is better to choose an entity type with total participation in R in the role of S

- Include the simple attributes of the 1:1 relationship type R as attributes of S.

If both participations are total, we may merge the two entity types and the relationship into a single relation.

Step 3: For each binary 1:1 relationship type R

Binary 1:1



- Case 1: Combination of participation types

The primary key of the partial participant will become the foreign key of the total participant

Employee(E#, Name,...)

Department (Dept#, Name...,MgrE#)



Step 3: For each binary 1:1 relationship type R

Binary 1:1



- Case 2: Uniform participation types

The primary key of either of the participants can become a foreign key in the other

Employee (E#, name...)

Chair (item#, model, location, used_by)

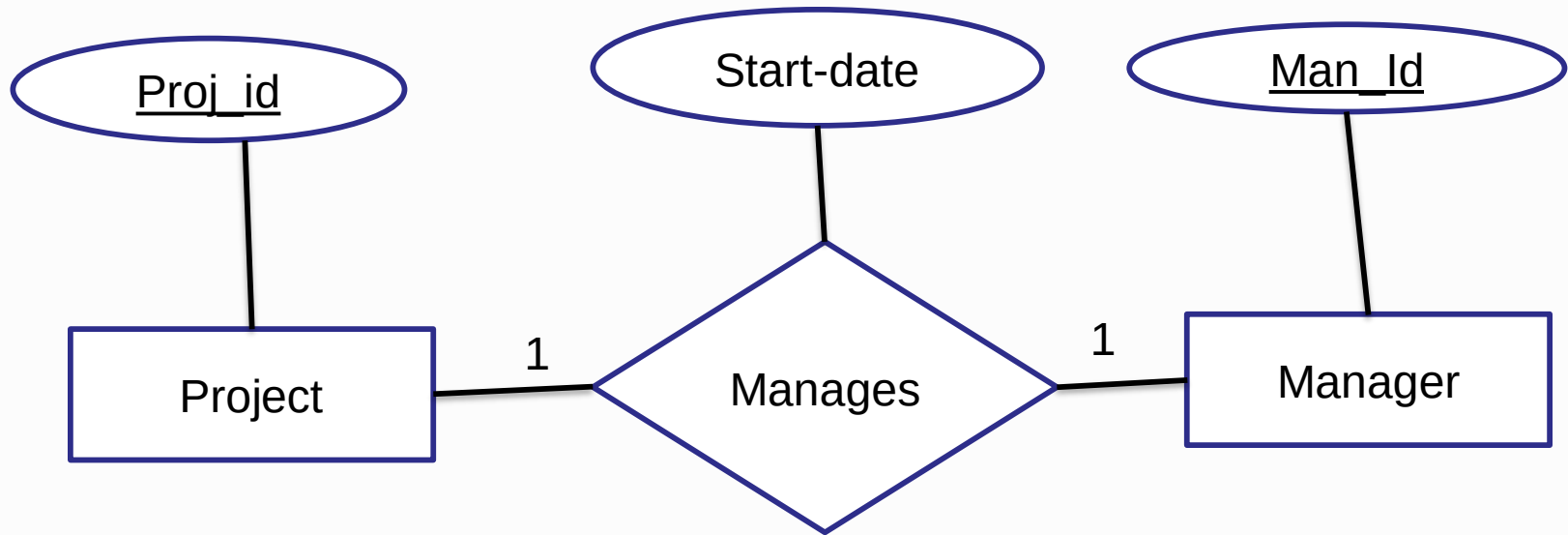
(or)

Employee (E#, Name....Sits_on)

Chair (item#,....)

Step 3: For each binary 1:1 relationship type R

Manages(Proj_id, Man_id, Start-date)



Step 4: For each regular binary 1:N relationship type

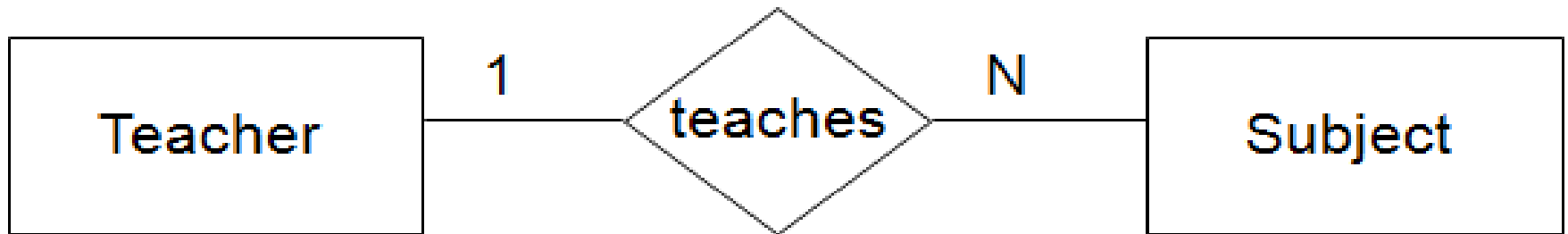
Identify the relation S that represents the participating entity type at the N-side of the relationship type.

Include as foreign key in S the primary key of the relations T that represents the other entity type participating in R .

Include any simple attributes of the 1:N relationship type as attributes of S .

Step 4: For each regular binary 1:N relationship type

Binary 1:N



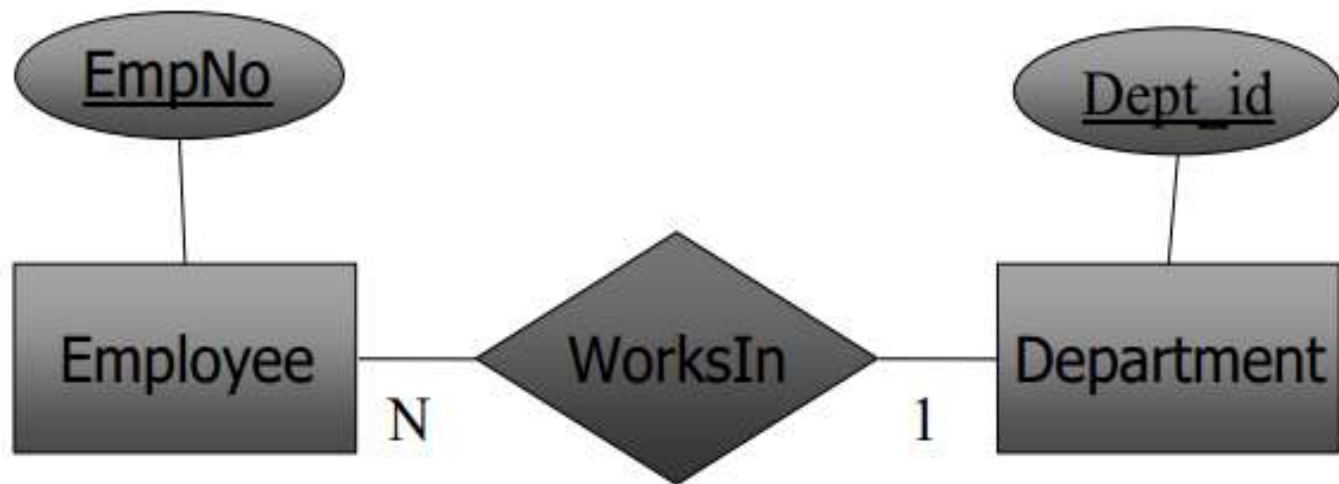
The primary key of the relation on the "1" side of the relationship becomes a foreign key in the relation on the "N" side

Teacher (ID, Name, Telephone, ...)

Subject (Code, Name, ..., Teacher)



Step 4: For each regular binary 1:N relationship type



Employee (EmpNo, Dept_id, ...)

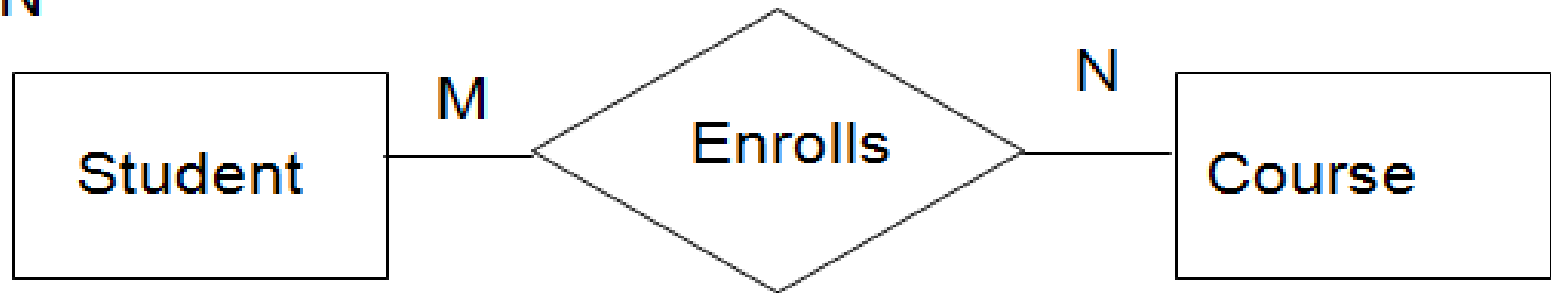
Step 5: For each binary M:N relationship type R

Create a new relation S to represent R.

- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types; their combination will form the primary key of S.
- Also, include any simple attributes of the
- M:N relationship type as attributes of S.

Step 5: For each binary M:N relationship type R

Binary M:N



- A new table is created to represent the relationship
- Contains two foreign keys - one from each of the participants in the relationship
- The primary key of the new table is the combination of the two foreign keys

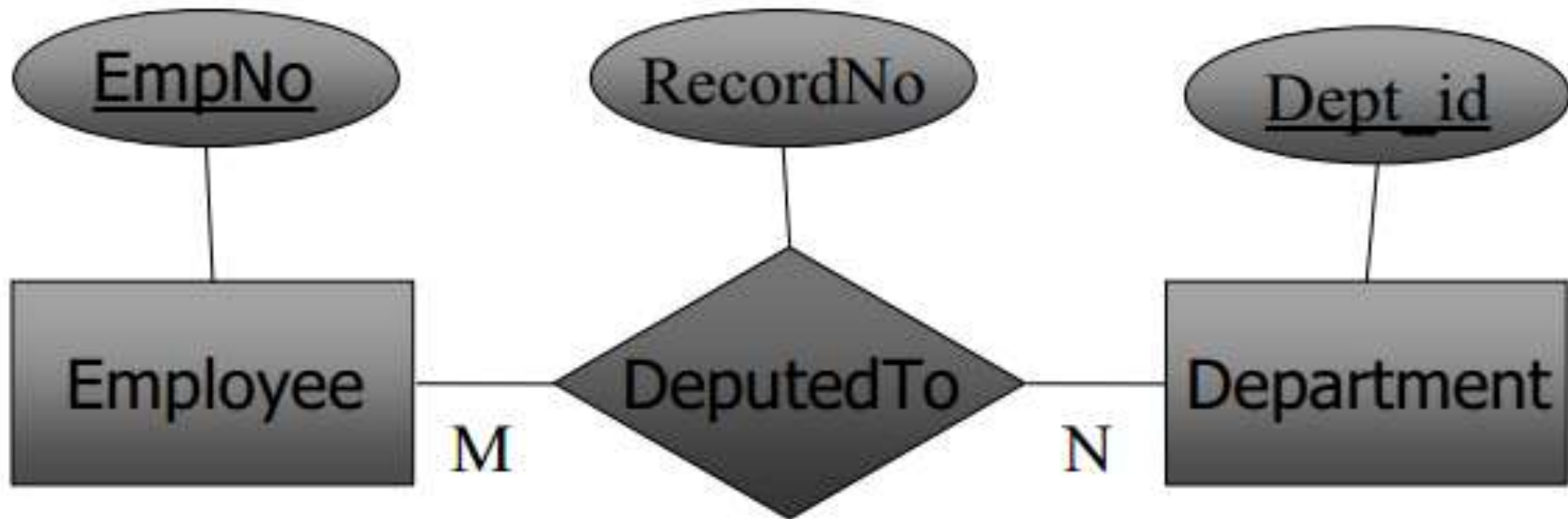
Student (Sid#, Title...)

Course(C#, CName,...)

Enrolls (Sid#, C#)



Step 5: For each binary M:N relationship type R



Employee (EmpNo, ...)

Department (Dept_id, ...)

DeputedTo (EmpNo, Dept_id, RecordNo, ...)

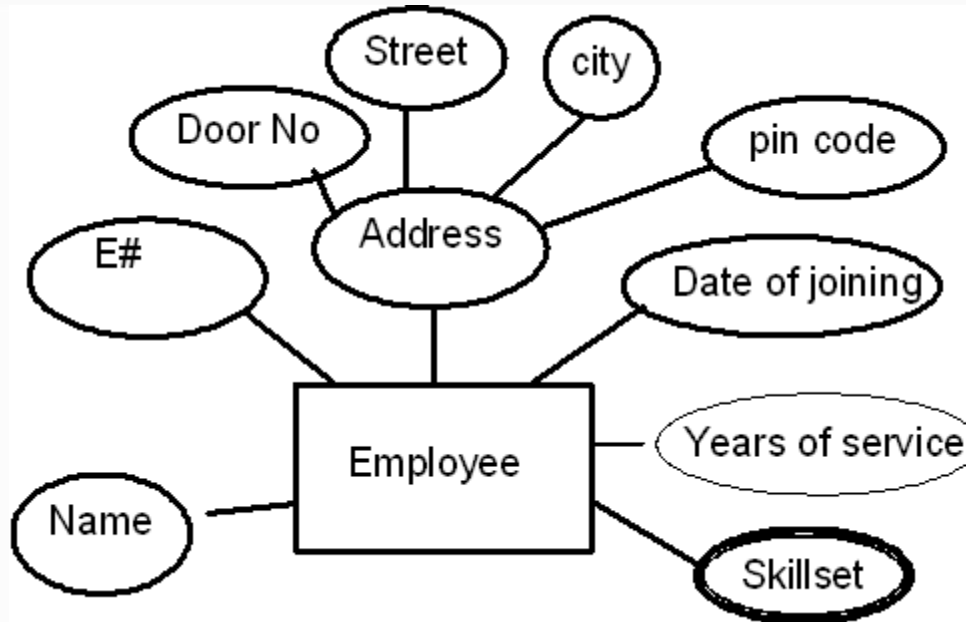
Step 6: For each multi-valued attribute A

Create a new relation R that includes

an attribute corresponding to A plus the primary key attribute K (as a foreign key in R) of the relation that represents the entity type or relationship type that has A as an attribute.

The primary key of R is the combination of A and K. If a multi-valued attribute is composite, we include its components.

Step 6: For each multi-valued attribute A



Employee (E#, Name, Door_No, Street, City, Pincode, Date_Of_Joining)

Emp_Skillset(E#, Skillset)

Step 7: For each n-ary relationship type R, $n > 2$

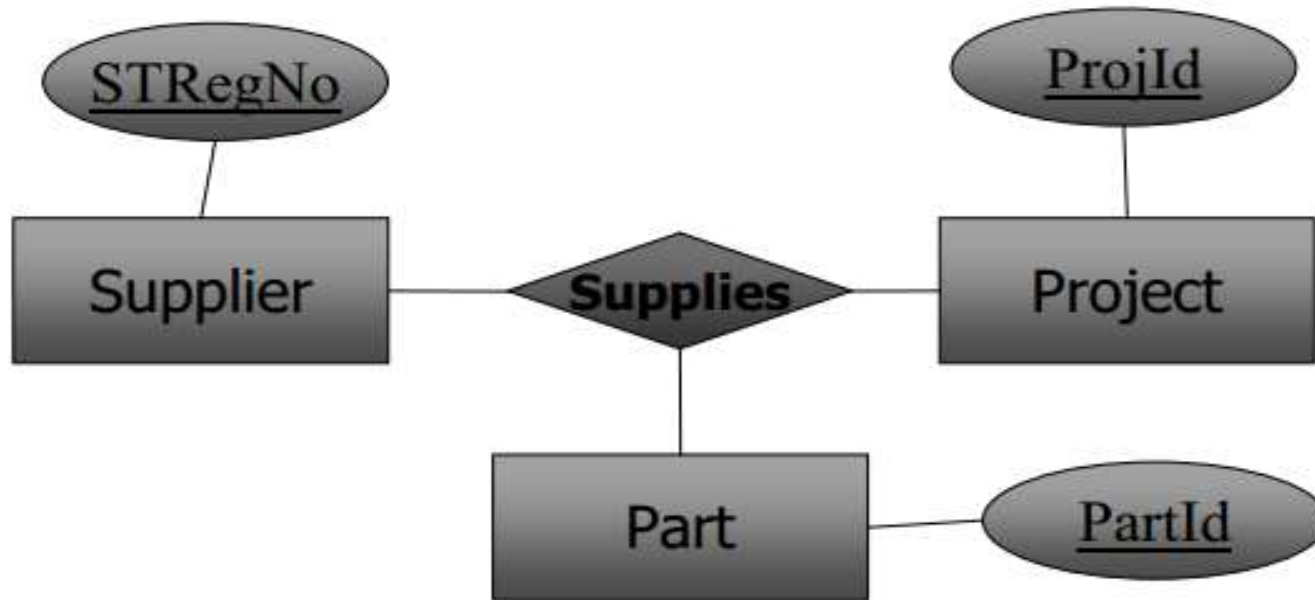
Create a new relation S to represent R.

Include as foreign key attributes in the S the primary keys of the relations that represent the participating entity types.

Also include any simple attributes of the n ary relationship types as attributes of S.

The primary key for S is usually a combination of all the foreign keys that reference the relations representing the participating entity types.

Step 7: For each n-ary relationship type R,
 $n > 2$

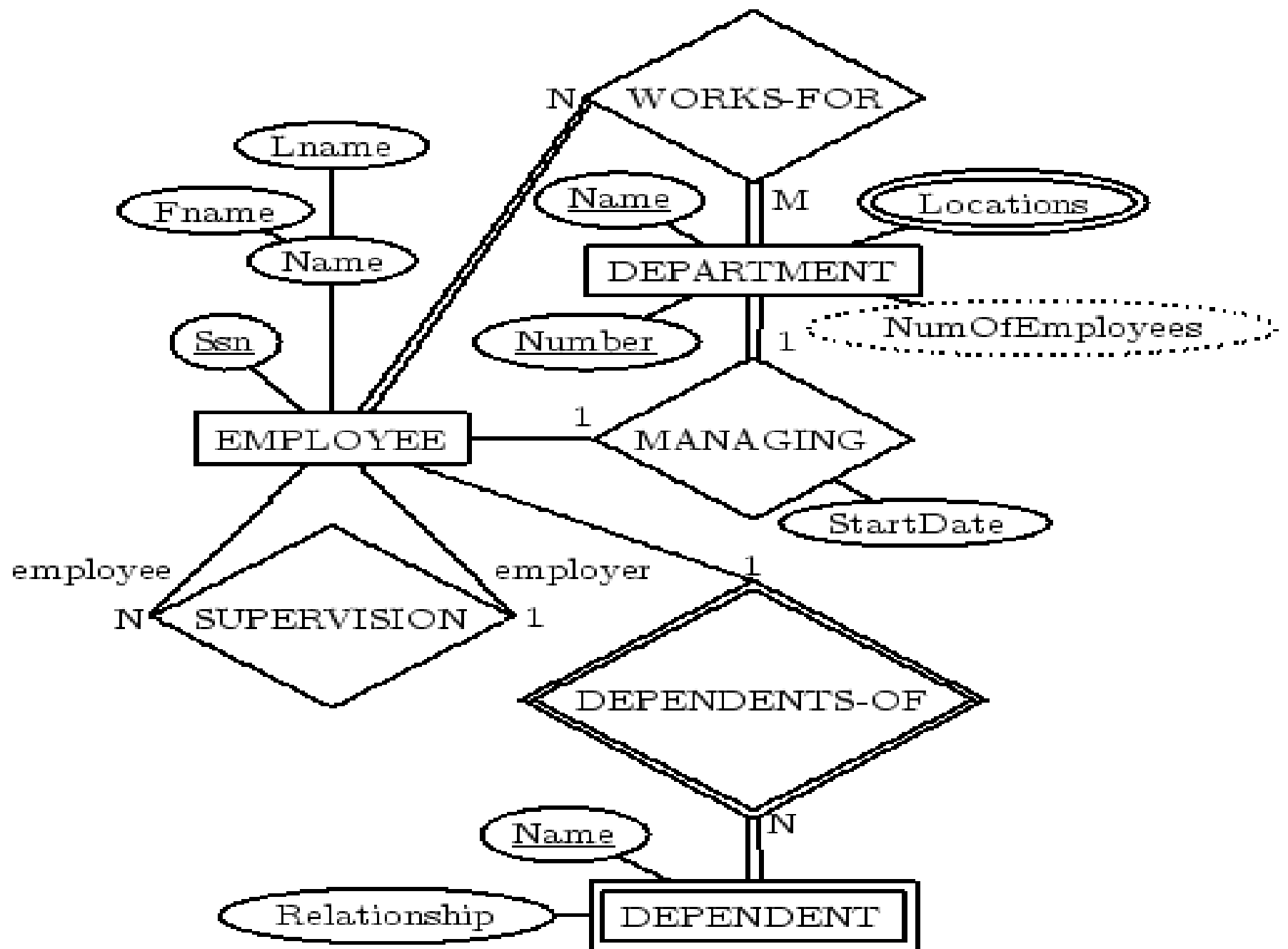


Supplier (STRegNo, ...)

Project (ProjId, ...)

Part (PartId, ...)

Supplies (STRegNo,
ProjId,
PartId)



EMPLOYEE

Fname	Lname	<u>SSN</u>	SupervisorSSN
-------	-------	------------	---------------

DEPARTMENT

Name	<u>NUMBER</u>	MANAGER-SSN	StartDate
------	---------------	-------------	-----------

DEPENDENT

Relationship	<u>EMPL-SSN</u>	<u>Name</u>
--------------	-----------------	-------------

DEP-LOCATION

Location	DEP-NUMBER
----------	------------

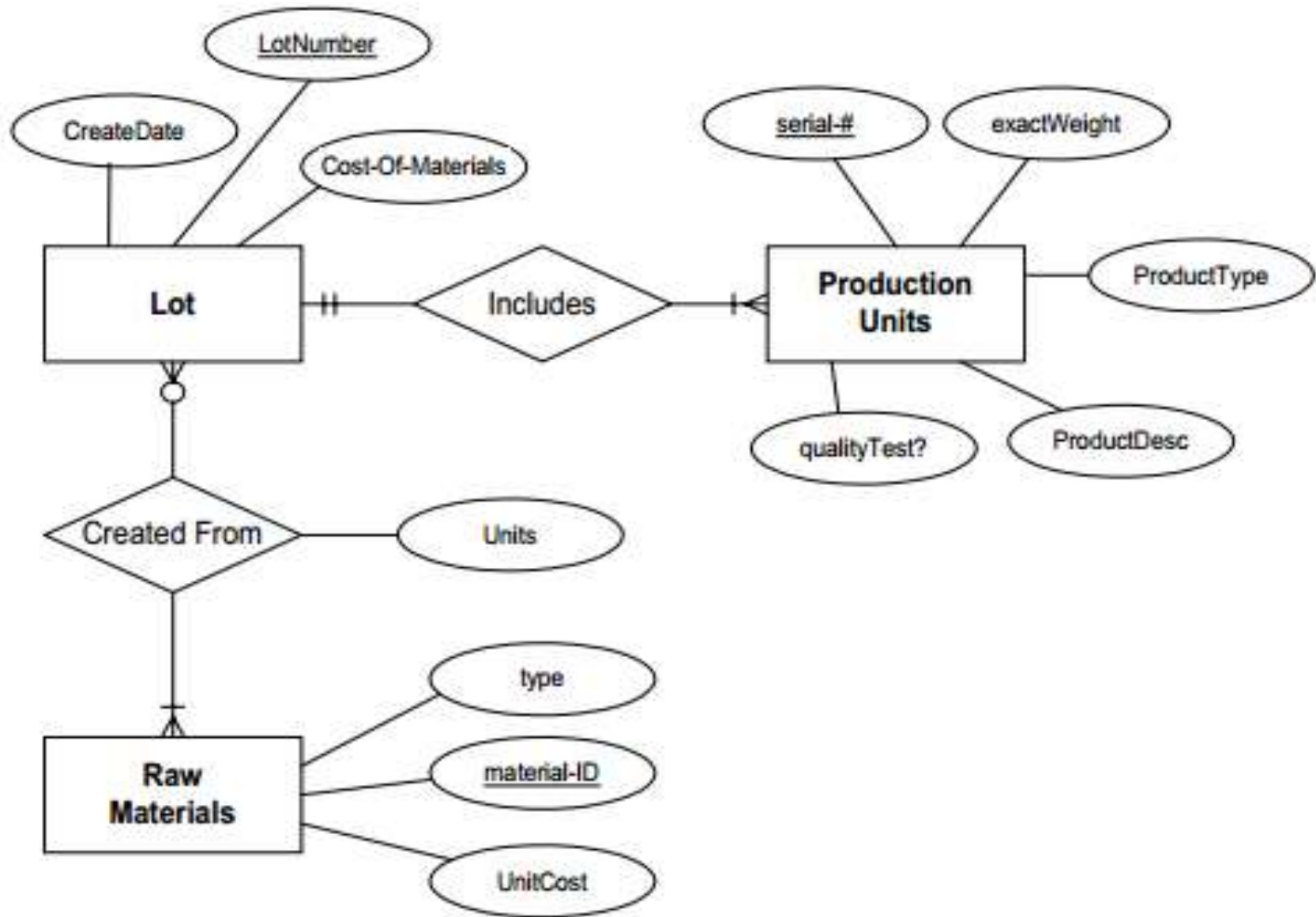
WORKS-FOR

EmployeeSSN	DeptNumber
-------------	------------

ER Model Relational Model

- Entity type- “Entity” relation
- 1:1 or 1:N relationship type -Foreign key (or “relationship” relation)
- M:N relationship type -“Relationship” relation and two foreign keys
- n-ary relationship type -“Relationship” relation and n foreign keys
- Simple attribute -Attribute
- Composite attribute- Set of simple component attributes
- Multivalued attribute- Relation and foreign key

Example1



Example1

Solutions:

Production Units

<u>Serial#</u>	ExactWeight	ProductType	ProductDesc	QualityTest?	<u>LotNumber</u>
----------------	-------------	-------------	-------------	--------------	------------------

Lot

<u>LotNumber</u>	CreateDate	CostOfMaterials
------------------	------------	-----------------

Raw Materials Usage

<u>LotNumber</u>	<u>MaterialID</u>	Units
------------------	-------------------	-------

Raw Materials

<u>MaterialID</u>	Type	UnitCost
-------------------	------	----------

EER to Relational Mapping

Step 8: To convert each super-class/subclass relationship into a relational schema you must use one of the four options available.

Let C be the super-class,
 K its primary key and
 $A_1, A_2, \dots, A_n$ its remaining attributes and
let $S_1, S_2, \dots, S_m$ be the sub-classes.

Option 8A (multiple relation option)

Create a relation L for C with attributes $\text{Attrs}(L) = \{K, A_1, A_2, \dots, A_n\}$ and $\text{PK}(L) = K$.

Create a relation L_i for each subclass S_i , $1 < i < m$, with the attributes $\text{ATTRS}(L_i) = \{K\} \cup \{\text{attributes of } S_i\}$ and $\text{PK}(L_i) = K$.

This option works for any constraints: disjoint or overlapping; total or partial.

Form a table for the superclass and form a table for each subclass. Include the primary key attributes of the superclass in each such table.

Example:

- Employees(SocialSN, Name, Address)
- Hourly_Emps(SocialSN, Hours, Wages)
- Contract_Emps(SocialSN, ContractNo)

Option 8A (multiple relation option)



EMPLOYEE

<u>SSN</u>	FName	MInit	LName	BirthDate	Address	JobType
------------	-------	-------	-------	-----------	---------	---------

SECRETARY

<u>SSN</u>	TypingSpeed
------------	-------------

TECHNICIAN

<u>SSN</u>	TGrade
------------	--------

ENGINEER

<u>SSN</u>	EngType
------------	---------

Option 8B (multiple relation option)

Create a relation L_i for each subclass S_i , $1 < i < m$, with $ATTRS(L_i) = \{\text{attributes of } S_i\} \cup \{K, A_1, A_2, \dots, A_n\}$
 $PK(L_i) = K$

This option works well only for disjoint and total constraints.

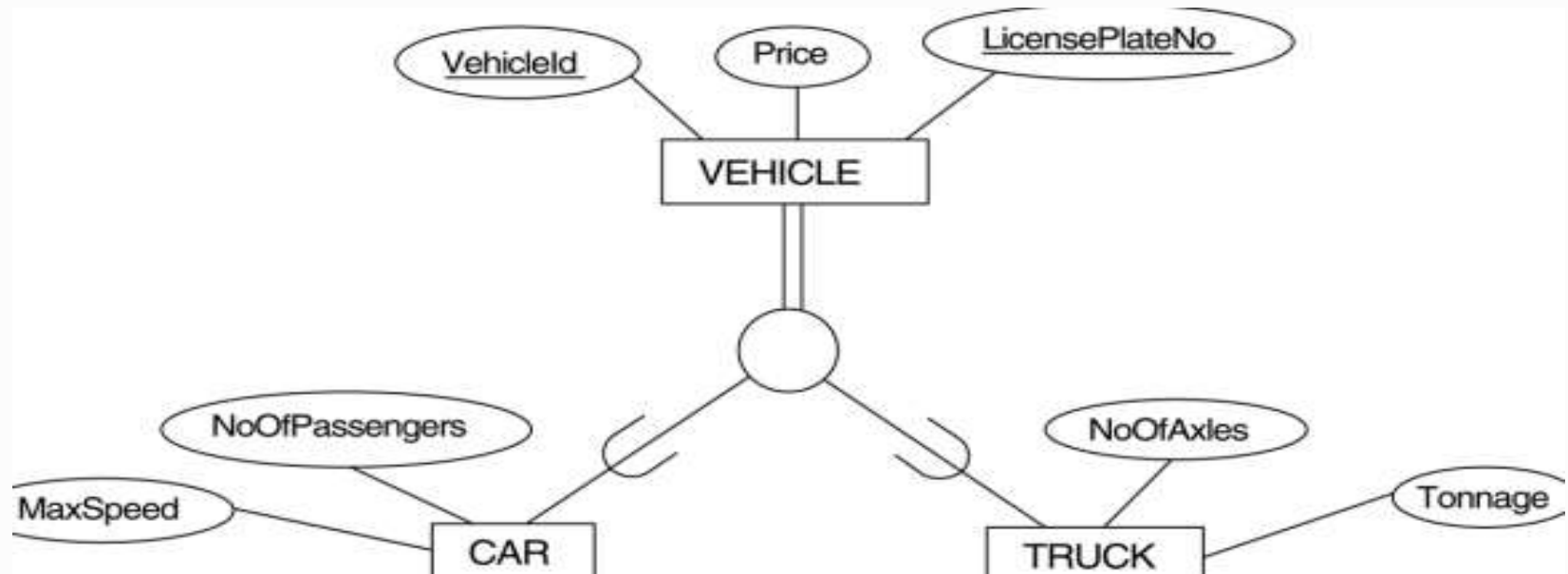
- If not disjoint, redundant values for inherited attributes.
- If not total, entity not belonging to any sub-class is lost.

Form a table for each subclass and include all attributes of the superclass.

- Hourly_Emps(SocialSN, Name, Address, Hours, Wages)
- Contract_Emps(SocialSN, Name, Address, Contractno)

Method 2 has no table for the superclass EMPLOYEES.

Option 8B (multiple relation option)



CAR

<u>VehicleId</u>	LicensePlateNo	Price	MaxSpeed	NoOfPassengers
------------------	----------------	-------	----------	----------------

TRUCK

<u>VehicleId</u>	LicensePlateNo	Price	NoOfAxles	
------------------	----------------	-------	-----------	--

Option 8C: Single relation with one type attribute

- Create a single relation L with attributes $\text{Attrs}(L) = \{k, a_1, \dots, a_n\} \cup \{\text{attributes of } S_1\} \cup \dots \cup \{\text{attributes of } S_m\} \cup \{t\}$ and $\text{PK}(L) = k$.
- The attribute t is called a type (or discriminating) attribute that indicates the subclass to which each tuple belongs

Use null values

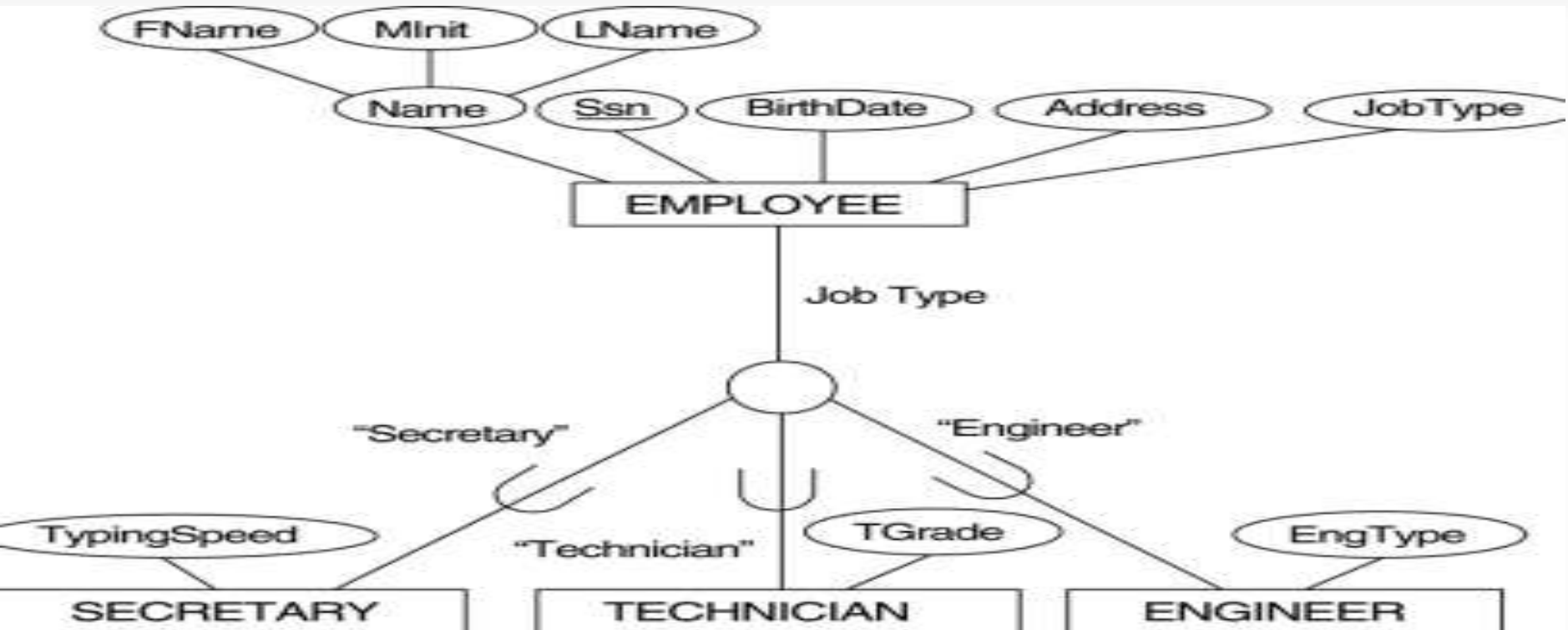
Employees(SocialSN, Name, Address, Hours, Wages, ContractNo)

Hourly employees will have a null value for ContractNo

.

Contract employees will have null values for Hours and Wages

Option 8C: Single relation with one type attribute



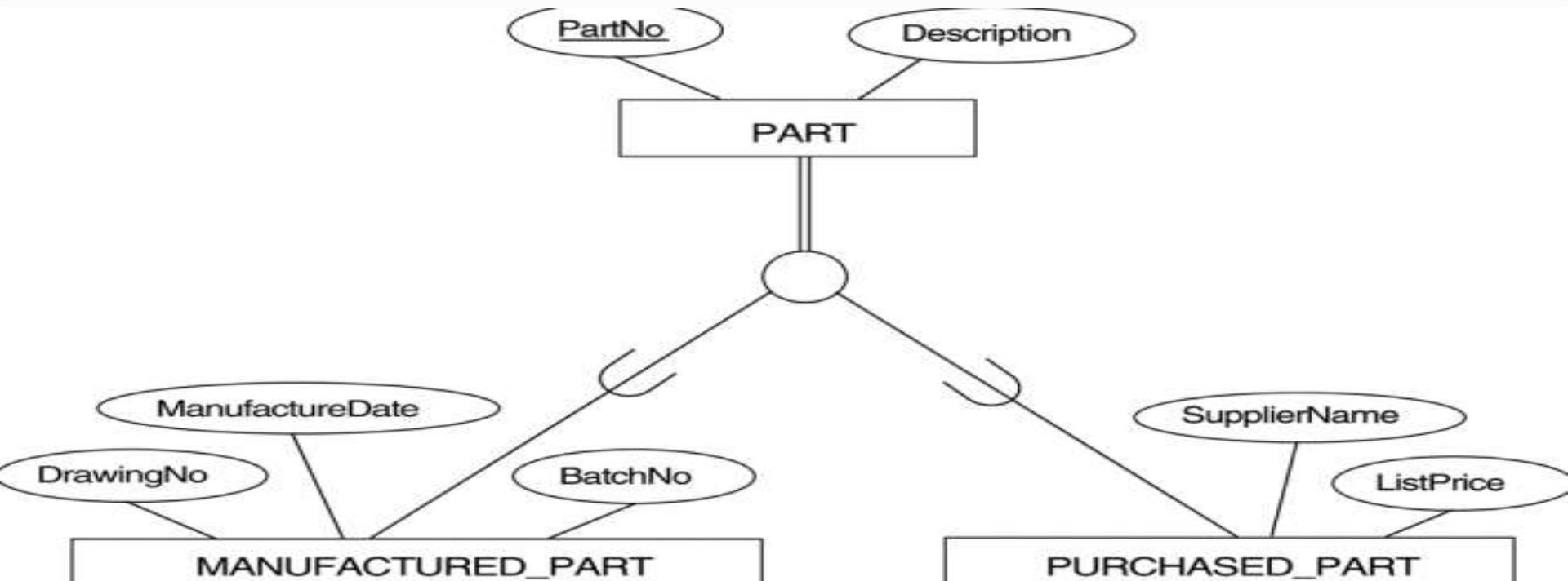
EMPLOYEE

<u>SSN</u>	FName	MInit	LName	BirthDate	Address	JobType	TypingSpeed	TGrade
------------	-------	-------	-------	-----------	---------	---------	-------------	--------

Option 8D: Single relation with multiple type attributes

Create a single relation schema L with attributes $\text{Attrs}(L) = \{k, 1, \dots, n\} \cup \{\text{attributes of } S_1\} \cup \{\text{attributes of } S_m\} \cup \{t_1, t_2, \dots, t_m\}$ and $\text{PK}(L) = k$. Each t_i , $1 \leq i \leq m$, is a Boolean type attribute indicating whether a tuple belongs to the subclass S_i .

Option 8D: Single relation with multiple type attributes



PART

<u>PartNo</u>	Description	MFlag	DrawingNo	ManufactureDate	BatchNo	PFlag	SupplierName	ListPrice
---------------	-------------	-------	-----------	-----------------	---------	-------	--------------	-----------

